

Estructuras de Datos: Comparación entre la Teoría y la Observación Experimental

Alexa Alpízar Mora, C20281 alexa.alpizar@ucr.ac.cr

Resumen—En este trabajo se analizaron cuatro estructuras de datos: lista simplemente enlazada, árbol de búsqueda binaria, árbol rojinegro y tabla hash para registrar cuánto tardan en realizar las operaciones de búsqueda y eliminación, para luego comparar los resultados experimentales con los tiempos que se esperan teóricamente. La metodología consistió en realizar tres corridas por estructura, registrando tiempos promedios. Los resultados mostraron que la lista enlazada, tuvo los peores tiempos, mientras que la tabla hash fue la más rápida. Los árboles binario y rojo-negro presentaron tiempos acorde a $n(\log n)$, aunque el árbol binario con datos ordenados mostró un comportamiento atípico debido a una optimización en su implementación. Se concluye que los datos experimentales validan las cotas teóricas, destacando la importancia del balanceo en árboles y la solución de colisiones de las tablas hash, mientras que las diferencias observadas en ciertos casos se explican por ajustes específicos en las estructuras.

Palabras clave—lista enlazada, árbol binario, árbol rojo-negro, tabla hash, complejidad algorítmica, experimentación.

Palabras clave—ordenamiento, selección, inserción, mezcla

I. INTRODUCCIÓN

Las estructuras de datos son fundamentales en el campo de las ciencias de la computación; se refieren a formas de organizar y almacenar datos, de manera que se puedan realizar operaciones sobre ellos de forma eficiente. En esta ocasión, nos centraremos en las listas enlazadas, los árboles de búsqueda binaria, así como los árboles rojinegro y las tablas hash; para esta tarea programada. Las listas enlazadas se componen de nodos, cada uno con un valor y un enlace al siguiente nodo, así como también al nodo anterior en el caso de las listas doblemente enlazadas. Los árboles de búsqueda binaria son estructuras jerárquicas donde cada nodo tiene como máximo dos hijos, y los valores de los hijos son menores o mayores que el valor del padre. Los árboles rojinegro, por otro lado, son una variación y además con restricciones de color en los nodos, lo que permite garantizar el equilibrio. Por último, las tablas hash utilizan funciones para mapear claves a índices en una matriz.

La búsqueda y eliminación varían su funcionamiento según la estructura de datos a utilizar. En una lista enlazada simple, ambas operaciones requieren recorrer secuencialmente los nodos. En un árbol binario de búsqueda, la búsqueda y eliminación siguen el orden de las llaves. En cambio, un árbol rojinegro mantiene el balance mediante reglas de color y por último, en una tabla hash, la búsqueda y eliminación se realizan gracias a la función hash y pueden haber colisiones.

El objetivo principal es implementar algunas de las estructuras de datos estudiadas en el curso, anteriormente mencionadas. Se recolectará información sobre el desempeño de

los algoritmos por medio de su implementación en código, y posteriormente a través de las pruebas de ejecución. Esta tarea tiene como propósito realizar experimentos con dichas estructuras y sus correspondientes operaciones, con el fin de analizar los resultados, reflexionar y comparar la eficiencia teórica de las estructuras de datos y sus algoritmos con la eficiencia observada durante los experimentos.

II. METODOLOGÍA

Para lograr lo propuesto, se llevaron a cabo una serie de operaciones con el fin de luego registrarlas. Se realizaron las mismas operaciones: insertar, buscar y eliminar, para cada una de las estructuras; sin embargo, con dos muestras distintas (datos ordenados y aleatorios).

Para la inserción, se utilizó una muestra de tamaño $n = 1,000,000$ cantidad de nodos. En la inserción aleatoria, las llaves son enteros seleccionados al azar en el rango $[0, 3n]$, y en la inserción ordenada, las llaves son enteros en orden ascendente. Luego, la siguiente operación fue la búsqueda de $e = 10,000$ elementos cuyo valor fue seleccionado al azar en el mismo rango $[0, 3n]$, independientemente del tipo de muestra que se tratara (ordenada o desordenada), y se registró el tiempo de duración del total de búsquedas realizadas, incluyendo los casos en que el elemento está en la lista y los casos en que no está. Por último, se realizó la eliminación de $e = 10,000$ elementos, cuyo valor es seleccionado al azar en el mismo rango $[0, 3n]$, independientemente de la muestra, y se registró el tiempo de duración del total de eliminaciones realizadas, incluyendo el caso en que el elemento está en la lista y es eliminado, y los casos en que no está.

Para recolectar los datos de las listas simplemente enlazadas, con dos muestras, se realizaron tres veces cada operación y se registró el tiempo de duración de cada corrida y su respectivo promedio. Dicho proceso de recolección se aplicó de la misma manera para los árboles de búsqueda binaria, rojinegros y tablas hash. Es importante resaltar que insertar n llaves ordenadas en un árbol de búsqueda binaria puede tomar demasiado tiempo si n es grande, por lo que para evitar la larga espera, se creó un método que producía un árbol donde se insertan las llaves en orden, pero sin usar el método inserción. También para llevar a cabo el experimento. [1] El código que se utilizó para la implementación de las estructuras de datos, tanto como el de las operaciones aplicadas está basado en el pseudocódigo del libro de Cormen y colaboradores [1].

III. RESULTADOS

Los tiempos de ejecución de las tres corridas de las operaciones, con cada estructura de datos, se muestran en los

cuadros I, II, III y IV.

Cuadro I: Tiempos de ejecución de la Lista Simplemente Enlazada

Operación	Muestra	Tiempo (s) por corrida			Promedio (s)
		1	2	3	
Búsqueda	Aleatoria	39.9691	38.7546	39.6232	39.4490
	Ordenada	39.2307	41.3404	44.0254	41.5322
Eliminación	Aleatoria	40.6132	41.5271	42.1332	41.4245
	Ordenada	43.3753	46.4258	45.3195	45.0402

Cuadro II: Tiempos de ejecución del Árbol Binario de Búsqueda

Operación	Muestra	Tiempo (s) por corrida			Promedio (s)
		1	2	3	
Búsqueda	Aleatoria	0.0136	0.0145	0.0125	0.0135
	Ordenada	0.0032	0.0031	0.0032	0.0032
Eliminación	Aleatoria	0.0137	0.0167	0.0148	0.0151
	Ordenada	0.0036	0.0029	0.0029	0.0032

Cuadro III: Tiempos de ejecución del Árbol Rojo-Negro

Operación	Muestra	Tiempo (s) por corrida			Promedio (s)
		1	2	3	
Búsqueda	Aleatoria	0.0102	0.0118	0.0101	0.0107
	Ordenada	0.0053	0.0053	0.0052	0.0053
Eliminación	Aleatoria	0.0125	0.0122	0.0141	0.0129
	Ordenada	0.0084	0.0086	0.0083	0.0085

Cuadro IV: Tiempos de ejecución de la Tabla Hash con Encadenamiento

Operación	Muestra	Tiempo (s) por corrida			Promedio (s)
		1	2	3	
Búsqueda	Aleatorios	0.00240	0.00223	0.00226	0.00229
	Ordenados	0.00208	0.00202	0.00195	0.00202
Eliminación	Aleatorios	0.00308	0.00278	0.00259	0.00281
	Ordenados	0.00226	0.00206	0.00204	0.00212

En los cuadros I, II, III y IV se pueden observar los resultados de cada estructura de datos al ejecutar búsqueda y eliminación. También, que se obtuvieron de dos muestras distintas para cada operación y se llevaron a cabo tres corridas con resultados diferentes, con los cuales se obtuvo su promedio correspondiente.

En el cuadro I, se obtuvo de la lista enlazada simple, un promedio de 39.4490s de duración para la operación de búsqueda al realizarse con la muestra de datos aleatorios y 41.5322s con la de datos ordenados. Mientras que en la eliminación se registró un promedio de 41.4245s de duración con la muestra aleatoria y 45.0402s con la ordenada.

Seguidamente, en el cuadro II, con la estructura de árbol binario se tuvo como resultado un promedio de duración de 0.0135s cuando se realizó la búsqueda con una muestra

aleatoria y 0.0032s con la ordenada. Para la operación de eliminación, los tiempos promedios fueron de 0.0151s con datos aleatorios y 0.0032s con datos ordenados.

En el cuadro III, correspondiente al árbol rojo-negro, los tiempos promedios de búsqueda fueron de 0.0107s para la muestra aleatoria y 0.0053s para la ordenada. En cuanto a la eliminación, se registraron promedios de 0.0129s (aleatoria) y 0.0085s (ordenada).

Por último, en el cuadro IV, que muestra los resultados de la tabla hash con encadenamiento, se observaron los menores tiempos de ejecución. Para la búsqueda, los promedios fueron de 0.00229s con datos aleatorios y 0.00202s con ordenados. En la eliminación, los tiempos promedios fueron de 0.00281s (aleatoria) y 0.00212s (ordenada).

Las diferenciaciones de duración, que se obtuvieron con respecto a las muestras, se pueden observar en las figuras 1-8, 1, 2, 3, 4, 5, 6, 7 y 8.

En la figura 9, los resultados muestran que la tabla hash registró el menor tiempo de ejecución con 0.00202 segundos en promedio para búsquedas con muestra ordenada. El árbol binario presentó un tiempo de 0.0032 segundos, seguido por el árbol rojinegro con 0.0053 segundos. La lista enlazada simple mostró el mayor tiempo de ejecución, con un promedio de 41.5322 segundos. Continuando con la figura 10, con datos desordenados, la tabla hash mantuvo el mejor rendimiento (0.00229 segundos). El árbol rojinegro registró 0.0107 segundos, superando ligeramente al árbol binario (0.0135 segundos). La lista enlazada continuó siendo la estructura más lenta, con un tiempo promedio de 39.4490 segundos.

Por otra parte en operaciones de eliminación con datos ordenados de la figura 11, la tabla hash destacó nuevamente con 0.00212 segundos. El árbol binario mostró un tiempo de 0.0032 segundos, mientras que el rojinegro registró 0.0085 segundos. La lista enlazada tuvo el peor desempeño, con un promedio de 45.0402 segundos. Por último, para eliminaciones con muestra desordenada de la figura 12, la tabla hash obtuvo 0.00281 segundos. El árbol rojinegro presentó un tiempo de 0.0129 segundos, siendo más rápido que el árbol binario (0.0151 segundos). La lista enlazada mantuvo los tiempos más altos, con 41.4245 segundos en promedio.

Los contrastes entre estructuras de datos, se pueden observar en las figuras: 9, 10, 11 y 12.

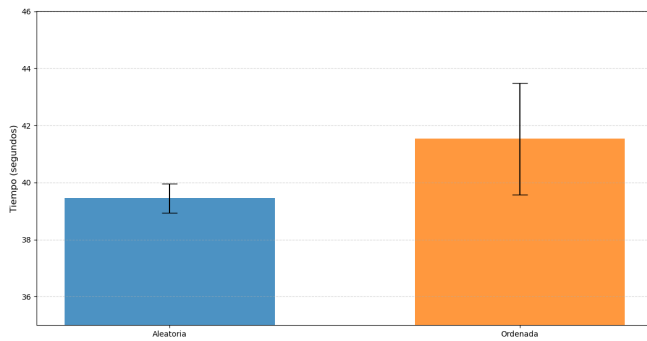


Figura 1: Gráfico Comparativo de Tiempos de Búsqueda de la Lista Enlazada Simple con Muestra Aleatoria y Ordenada.

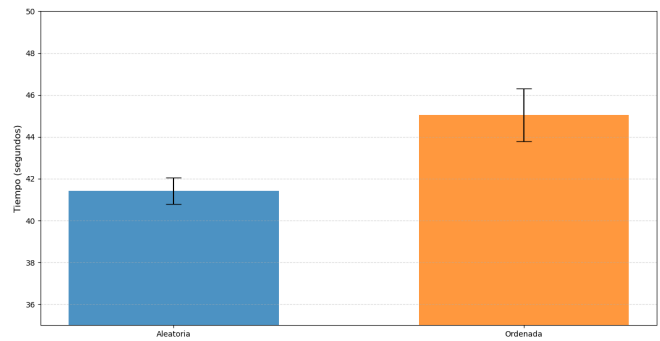


Figura 5: Gráfico Comparativo de Tiempos de Eliminación de la Lista Enlazada Simple con Muestra Aleatoria y Ordenada.

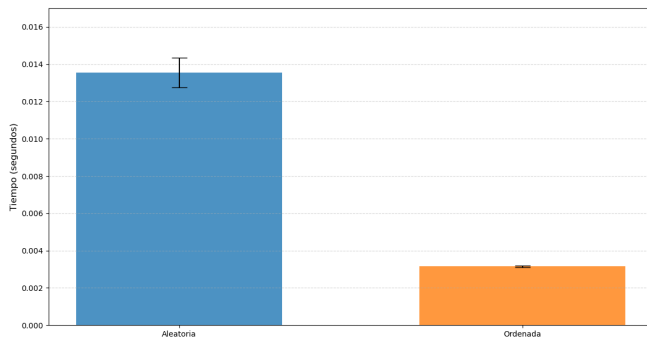


Figura 2: Gráfico Comparativo de Tiempos de Búsqueda del Árbol Binario con Muestra Aleatoria y Ordenada.

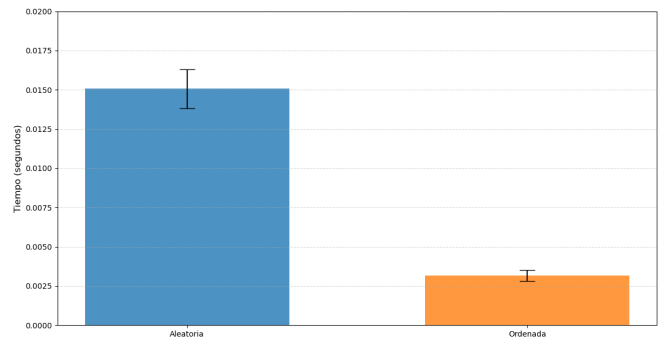


Figura 6: Gráfico Comparativo de Tiempos de Eliminación del Árbol Binario con Muestra Aleatoria y Ordenada.

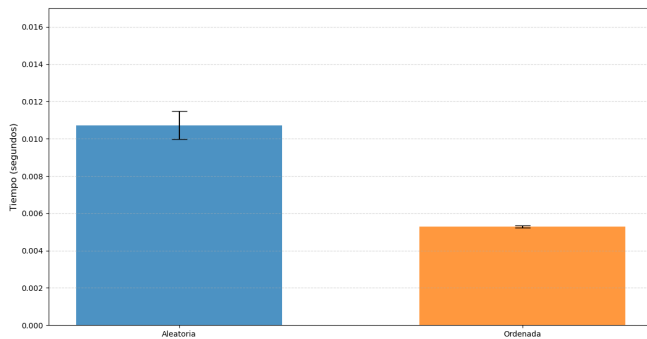


Figura 3: Gráfico Comparativo de Tiempos de Búsqueda del Árbol Rojinegro con Muestra Aleatoria y Ordenada.

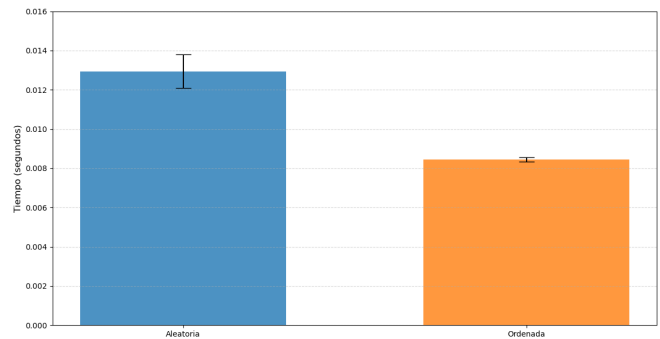


Figura 7: Gráfico Comparativo de Tiempos de Eliminación del Árbol Rojinegro con Muestra Aleatoria y Ordenada.

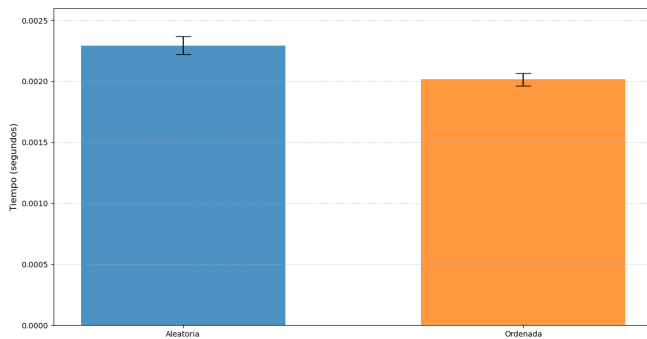


Figura 4: Gráfico Comparativo de Tiempos de Búsqueda de la Tabla Hash con Muestra Aleatoria y Ordenada.

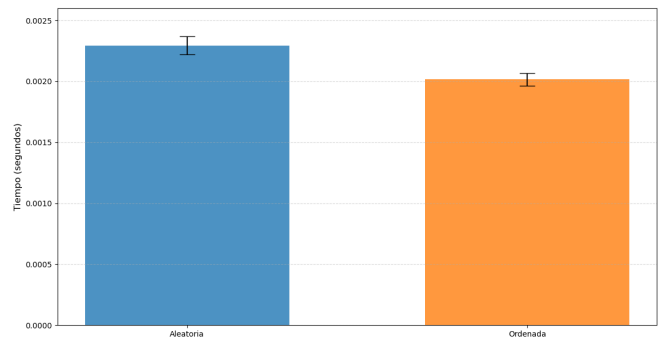


Figura 8: Tiempos de Eliminación de la Tabla Hash con Muestra Aleatoria y Ordenada.

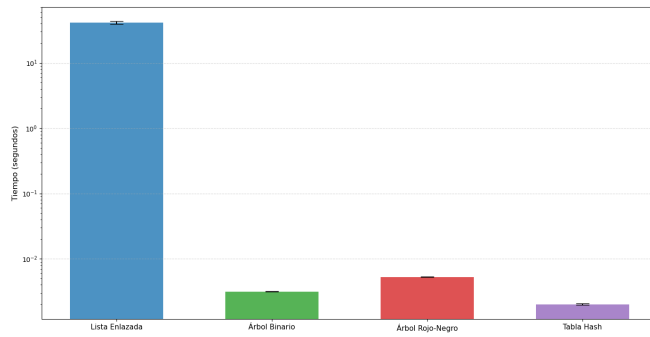


Figura 9: Comparativo de Tiempos de Búsqueda (Muestra Ordenada).

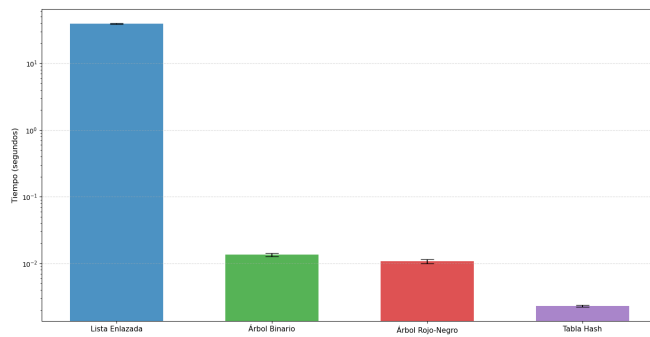


Figura 10: Comparativo de Tiempos de Búsqueda (Muestra Desordenada).

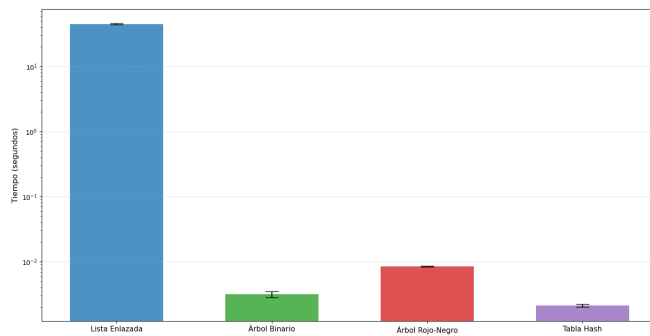
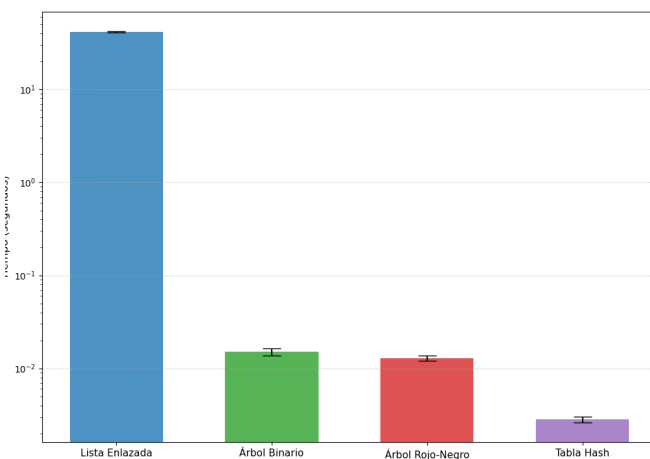


Figura 11: Comparativo de Tiempos de Eliminación (Muestra Ordenada).



IV. DISCUSIÓN

Como primer punto, con base en los resultados presentados en los cuadros I-IV, se puede observar que los tiempos de ejecución para las operaciones de búsqueda y eliminación varían significativamente entre las estructuras de datos analizadas. Estos resultados coinciden con las cotas teóricas esperadas según el libro Cormen et al.

En el caso de la lista simplemente enlazada, los tiempos reportados para búsqueda y eliminación, tanto con muestra de datos aleatorios como ordenados, se mantienen por los 40-45s. El resultado obtenido parece coherente con su complejidad teórica de $O(n)$ como el peor caso ya que para ambas muestras se requiere recorrer la lista secuencialmente. Las ligeras diferencias entre los tiempos con datos aleatorios y ordenados podrían deberse a cómo se accede a los elementos en la memoria o a las verificaciones para eliminar un nodo, así como como ajustar los punteros entre los nodos de la lista, sin embargo son tiempos altos y muy similares.

Adicionalmente, los experimentos realizados revelan que para el árbol búsqueda binaria, los resultados muestran tiempos considerablemente menores: alrededor de 0.0135 s para búsqueda aleatoria y 0.0151s para eliminación aleatoria. Estos tiempos parecen acercarse a la cota $O(\log n)$ que se espera en promedio cuando el árbol está balanceado. Con datos ordenados, los tiempos fueron aún menores (alrededor de 0.0032 s en ambas operaciones), lo cual podría parecer contradictorio con la teoría, ya que normalmente se esperaría que las operaciones duraran más ($O(n)$) si el árbol se convirtiera en una lista, debido al completo desbalanceo. Sin embargo, en este experimento se utilizó un método de inserción optimizado ("fastInsert") que construye el árbol de forma balanceada incluso con datos ordenados, lo que tal vez podría explicar por qué las operaciones no tardaron más, como se esperaba. En el caso del árbol rojinegro los resultados alrededor de 0.0107s (aleatorio) y 0.0035s (ordenado) también se asemejan a la duración $O(\log n)$ esperada, además de que los tiempos son consistentemente bajos, lo que ocurre cuando un árbol está balanceado.

Por otra parte, la tabla hash presenta los tiempos más bajos de todas las estructuras evaluadas, especialmente en las búsquedas ordenadas (alrededor de 0.00002 s). Estos resultados parecen alinearse con la complejidad $O(1)$ esperada para esta estructura en operaciones de acceso. Sin embargo, se puede observar cierta variación en los tiempos de eliminación; por ejemplo, 0.00081 s con datos aleatorios frente a 0.00012 s con datos ordenados, lo cual podría estar probablemente relacionado con cómo se resuelven las colisiones. En este experimento, las colisiones se resolvieron mediante listas doblemente enlazadas, lo que facilita la eliminación de elementos, aunque el tiempo que toma puede variar dependiendo de cuántos elementos se encuentren en la misma posición de la tabla.

V. CONCLUSIONES

Retomando los objetivos de este trabajo, se puede concluir que, luego de la experimentación y su comparación con la teoría estudiada en clase, efectivamente se observa

una similitud entre los resultados obtenidos y los esperados. En los casos donde no hay coincidencia, esto se debe a modificaciones específicas implementadas. Por ejemplo, en el árbol de búsqueda binaria, si no se hubiera aplicado un método optimizado para balancear los datos ordenados, la duración habría sido significativamente peor. Como se explica en la teoría, en el peor de los casos, especialmente en operaciones como búsqueda o eliminación, el árbol degeneraría en una estructura lineal (similar a una lista), con una profundidad excesiva, lo que aumentaría considerablemente el tiempo de recorrido. De igual manera, en la tabla hash, el uso de listas doblemente enlazadas para resolver colisiones fue clave para garantizar un acceso más rápido a los elementos en comparación a las demás estructuras. Sin esta estrategia, su duración muy posiblemente hubiera aumentado, especialmente en operaciones de eliminación. Finalmente, los resultados experimentales validan los conceptos teóricos de la lista enlazada simple y el árbol rojinegro, y las diferencias observadas en el caso del árbol de búsqueda binaria y la tabla hash se explican por las optimizaciones implementadas en las estructuras de datos.

REFERENCIAS

- [1] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed. The MIT Press, 2009.



Alexa Alpízar Mora Me gusta mucho el mar, los deportes, pasar tiempo con mis amigos y aprender sobre temas variados en la universidad.